

What does a $B^\pm$ candidate mass peak look like?

My first attempt at reading an LHCb ROOT file and fitting a Gaussian peak model

For my second data-analysis project, I wanted to move away from an ordinary tabular dataset and try working with data from CERN.

I used an LHCb ROOT ntuple containing reconstructed $B^\pm \rightarrow J/\psi K^\pm$ candidates. My immediate aim was deliberately narrow: understand the structure of the ROOT file, extract the `Bplus_MM` branch, plot its distribution, and attempt a simple Gaussian fit around the visible peak.

This was my first time working with a ROOT file. A large part of the project was therefore not the final graph, but understanding what a tree, branch and array actually represented.

Why this project?

My first project used the NASA Exoplanet Archive, which behaved mostly like a conventional table. For this project, I wanted to become familiar with a file structure commonly used in particle and high-energy physics.

I attempted to reproduce part of the output of the [REANA LHCb Run 2 \$B^\pm \rightarrow J/\psi K^\pm\$ example](#) provided alongside the CERN Open Data resources.

The original example plots the reconstructed $B^\pm$ and J/ψ candidate-mass distributions using ROOT's `RDataFrame`. My implementation was not an exact reproduction. I used `uproot`, `NumPy`, `Matplotlib` and `SciPy`, selected only the `Bp1us_MM` branch, and added a simple Gaussian-plus-background fit.

The question I used

Can I extract one mass branch from an unfamiliar ROOT file, visualise its distribution, and fit the visible peak using a simple statistical model?

Important limitation

This is a programming and data-analysis exercise

I will not attempt to explain the detailed particle physics behind the dataset or give a physical interpretation of every structure in the mass distribution.

I am not yet sufficiently well versed in LHCb reconstruction, detector effects, decay backgrounds or particle-physics fitting methods to make those claims responsibly.

The fitted results in this report should therefore be read as the output of a simple learning exercise, not as an independent scientific measurement.

My focus is on what I did computationally, what the fitted parameters mean within the chosen mathematical model, and what I learned while working with the ROOT format.

The data

I used the CERN Open Data dataset

$$B^\pm \rightarrow J/\psi(\rightarrow \mu^+ \mu^-) K^\pm$$

Ntuples 4530 — 2017 Magnet Up.

It contains data from proton-proton collisions recorded by the LHCb experiment and filtered into ntuples for exploring this decay channel.

The dataset can be found at:

[CERN Open Data record 93949](#)

The parent ntuple collection is cited by CERN Open Data as:

[10.7483/OOPENDATA.LHCB.EBYF.C7OY](https://opendata.cern.ch/record/10.7483/OOPENDATA.LHCB.EBYF.C7OY)

Table 1: Scope of the data used in this project.

Item	Choice used here
Experiment	LHCb
Data-taking period	2017
Magnet configuration	Magnet Up
Ntupling request	4530
ROOT tree	Btree/DecayTree
Branch analysed	Bplus_MM
Main output	Candidate-mass histogram and Gaussian-plus-constant fit

Understanding the ROOT file

Opening the file produced the following top-level objects:

- Btree
- Btree/DecayTree
- GetIntegratedLuminosity
- GetIntegratedLuminosity/LumiTuple

The important object for this analysis was Btree/DecayTree.

The mental model that finally made the structure understandable was:

ROOT file → **tree** → **branch** → **numerical array**

The tree behaves somewhat like a table:

- an **entry** is similar to a row,
- a **branch** is similar to a column,
- and each branch stores one particular reconstructed quantity.

However, a ROOT tree can contain structures more complicated than an ordinary spreadsheet column. Some branches may contain vectors, matrices or arrays rather than one scalar value per entry.

This became clear when I attempted to convert the entire tree into a pandas DataFrame and received:

```
ValueError: Per-column arrays must each be 1-dimensional
```

Instead of forcing the complete tree into a DataFrame, I decided to read only the scalar branch needed for this analysis.

Why the branch was converted into a NumPy array

Selecting `tree["Bplus_MM"]` returns a ROOT branch object. It represents the stored branch and knows how to access its contents.

Reading it as a NumPy array loads and decompresses the actual numerical mass values so they can be passed to NumPy, Matplotlib and SciPy.

Analysis choices

Table 2: Decision log for the analysis.

Part	Choice	Value or method	Reason
File access	Python library	uproot	Allowed the ROOT file to be read directly in Python without using the full ROOT framework.
Data structure	Tree	Btree/DecayTree	This contained the reconstructed candidate variables.
Variable	Mass branch	Bplus_MM	I restricted the first exploration to one understandable scalar quantity.
Data container	NumPy array	One-dimensional mass array	Avoided the multidimensional branches that prevented complete DataFrame conversion.
Initial histogram	Bin spacing	Linear	The displayed mass range did not span several orders of magnitude.
Full view	Displayed range	Approximately 4500–6000 MeV/ c^2	Showed the complete stored distribution and the location of the dominant peak.
Fit view	Mass interval	Approximately 5200–5360 MeV/ c^2	Restricted the fit to the region surrounding the visible peak.
Peak model	Signal shape	Gaussian	Used as the simplest first approximation to the visible peak.

Part	Choice	Value or method	Reason
Background model	Background shape	Constant d	Used as a deliberately simple first background approximation.
Optimisation	Fitting function	<code>scipy.optimize.curve_fit</code>	Automatically adjusted the model parameters to match the histogram counts.
Initial centre	Starting guess	$\mu_0 = 5280 \text{ MeV}/c^2$	Chosen from the visible location of the peak.
Initial width	Starting guess	$\sigma_0 = 20 \text{ MeV}/c^2$	Chosen from a rough visual estimate of the peak width.

First look at the mass distribution

Before fitting anything, I calculated basic descriptive statistics for the complete `Bp1us_MM` array.

Table 3: Summary statistics of the complete `Bp1us_MM` array.

Statistic	Value (MeV/c^2)
Minimum	4773.543
First quartile	4955.390
Mean	5139.155
Median	5232.141
Third quartile	5283.724
Maximum	5779.302

The mean of the complete array is much lower than the location of the dominant peak. This showed why the mean of every stored candidate could not simply be treated as the centre of that peak.

The complete distribution contains the peak together with all other stored candidate values. The Gaussian fit was therefore performed only on a narrower interval surrounding the peak.

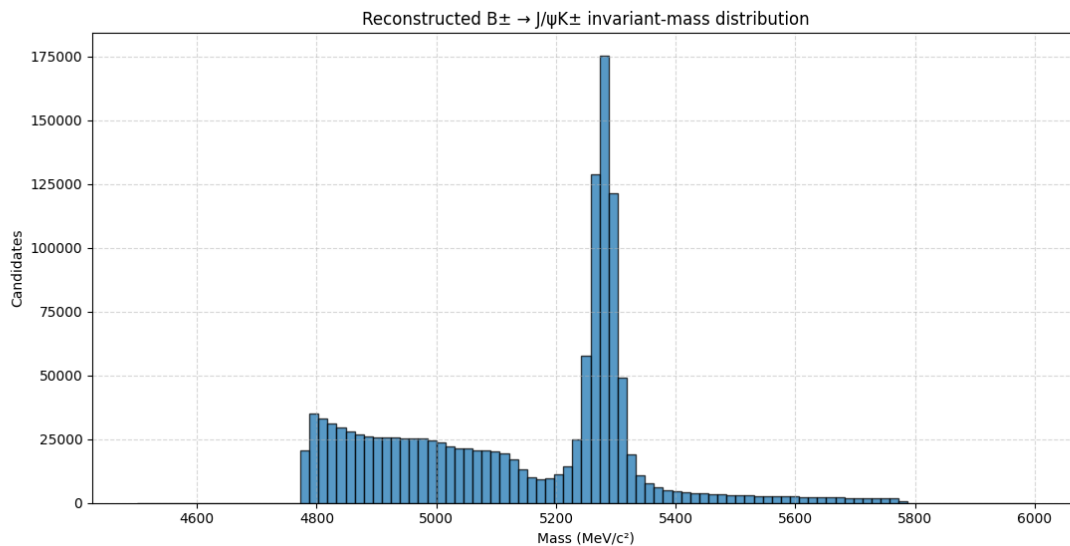


Figure 1: Full reconstructed $B^\pm$ candidate-mass distribution from the $B_{\text{p1us_MM}}$ branch. Linear bins were used because the values occupy a relatively narrow numerical range.

What I used the full plot for

The purpose of this plot was not to make a detailed physical interpretation. I used it to locate the dominant narrow peak, see that the complete distribution was not Gaussian, and decide that any simple peak fit should be restricted to a smaller mass interval.

Constructing the Gaussian fit

The model I used was

$$f(m) = A \exp \left[-\frac{(m - \mu)^2}{2\sigma^2} \right] + d.$$

The four parameters are:

Table 4: Parameters of the fitted model.

Parameter	Role in the model	What it controls
A	Gaussian amplitude	The height of the fitted Gaussian above the background.
μ	Gaussian centre	The horizontal location of the fitted peak.
σ	Gaussian standard deviation	The width of the fitted peak.
d	Constant term	The background level assumed by this simple model.

Turning the histogram into fit points

The original array contains individual candidate masses. A histogram converts those values into:

- a count for each bin, and
- the edges defining each bin.

The midpoint of each pair of neighbouring edges was then used as the horizontal coordinate of the corresponding bin.

The fit therefore compared pairs of the form

(bin centre, candidate count).

What `curve_fit` did

I supplied `curve_fit` with:

1. the mathematical model,
2. the bin centres,
3. the observed counts,
4. and initial guesses for A , μ , σ and d .

The initial guess for d was zero. This did **not** force the final background to be zero.

The fitting algorithm automatically varied all four parameters and returned the values that minimised the disagreement between the model and the histogram points.

The output `popt` contained the fitted parameters, while `pcov` contained their estimated covariance matrix.

Fit results

The optimal parameters returned by the fit were

$$[A, \mu, \sigma, d] = [15514.420, 5279.408, 17.753, 1138.890].$$

The uncertainties obtained from the square roots of the diagonal elements of the covariance matrix were

$$[75.005, 0.089, 0.115, 45.509].$$

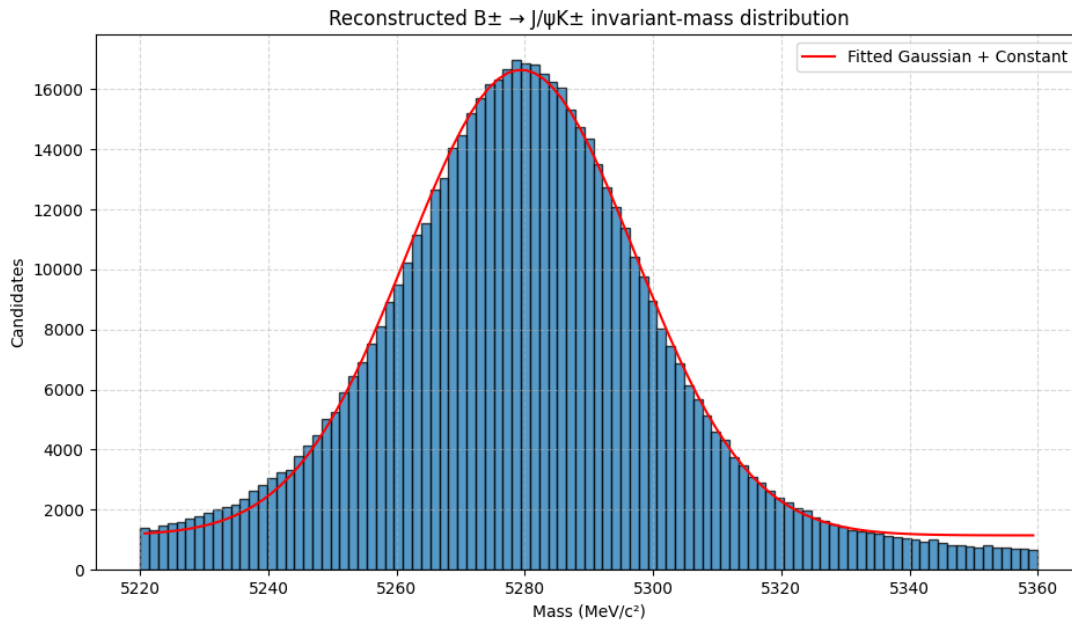


Figure 2: Gaussian-plus-constant model fitted to the candidate-mass histogram over the restricted peak region. The red curve is the fitted model.

FITTED PEAK CENTRE
 5279.408 ± 0.089
 MeV/c²

FITTED GAUSSIAN WIDTH
 17.753 ± 0.115
 MeV/c²

Table 5: Numerical output of the Gaussian-plus-constant fit.

Parameter	Meaning	Fitted value	Fit uncertainty
A	Gaussian height above the background	15514.420	75.005
μ	Peak centre in MeV/c ²	5279.408	0.089
σ	Gaussian width in MeV/c ²	17.753	0.115
d	Constant background in candidates per bin	1138.890	45.509

At the Gaussian centre, the model predicts a total height of approximately

$$A + d = 15514.420 + 1138.890 \approx 16653.31$$

candidates per bin.

The fitted full width at half maximum is

$$\text{FWHM} = 2\sqrt{2\ln 2}\sigma \approx 2.355\sigma \approx 41.81 \text{ MeV}/c^2.$$

What the fit does and does not establish

The main purpose of fitting the curve was to replace a visual statement such as

“There appears to be a peak near 5280.”

with numerical properties of that peak under a specified model.

Supported by this exercise	Not established by this exercise
The histogram contains a prominent peak that can be approximated by a Gaussian over a restricted range.	The complete mass distribution follows a Gaussian.
The fitted model places the peak centre at approximately $5279.4 \text{ MeV}/c^2$.	This is a complete precision measurement of the $B^\pm$ mass.
The fitted Gaussian width is approximately $17.75 \text{ MeV}/c^2$.	The width has been fully interpreted in terms of detector and particle physics.
The optimiser estimated a non-zero constant background level.	The true background is constant across the entire distribution.
The covariance matrix provides statistical uncertainties under the chosen fit model.	The quoted uncertainties include systematic effects, model uncertainty, binning choices or calibration uncertainty.

Why the quoted uncertainty must be treated carefully

The uncertainty of $0.089 \text{ MeV}/c^2$ on the fitted centre comes from the covariance matrix returned for this particular binned Gaussian-plus-constant fit. It does not include uncertainty caused by changing the binning, fit interval, background model, signal model, detector calibration or candidate selection. It should not be presented as the total uncertainty of a particle-mass measurement.

The constant-background model is visibly only a first approximation. I stopped the project here because the purpose was to become familiar with the workflow, not to construct a complete particle-physics fit.

A more developed analysis would compare alternative background functions, plot residuals, test the stability of the result under different fit ranges and bin widths, and investigate more appropriate signal models.

What I learned

1. A ROOT file is not simply a large spreadsheet. It contains trees and branches, and individual branches can store scalar or multidimensional structures.
2. A ROOT branch object is not yet the same thing as a NumPy array of its values. Reading the branch performs the step of loading and interpreting the stored data.
3. Converting an entire tree into a DataFrame is not always sensible. Selecting only the required branches can be simpler and more memory-efficient.
4. A histogram converts individual measurements into bin counts and bin edges. Bin centres can then be used as the x -coordinates for a binned curve fit.

5. Initial parameter values are only starting guesses. The optimiser automatically varies the parameters and returns its own fitted values.
6. Fitting a curve is useful because it produces numerical estimates for the peak centre and width rather than relying only on visual inspection.
7. A fitted result is meaningful only within its model. A Gaussian plus a constant is a mathematical approximation, not a complete description of the dataset.
8. A small covariance-derived uncertainty does not automatically mean that every source of uncertainty has been included.

AI use declaration

I used ChatGPT extensively as a tutor during this project.

This was my first time working with a ROOT file, and I initially did not understand the relationship between the file, tree, branches and arrays. I asked ChatGPT to explain the structure, help me understand error messages, and clarify what the different pieces of the Python workflow were doing.

The idea of fitting the visible peak with a Gaussian-plus-background model was suggested by ChatGPT because I did not yet know what kind of analysis would make sense for this dataset.

ChatGPT also helped me understand:

- why the mean and standard deviation of the complete mass array should not be used directly as the peak parameters,
- why a histogram PDF and candidate counts use different normalisations,
- how histogram counts, bin edges and bin centres are related,
- what `curve_fit`, `popt` and `pcov` represent,
- and how to organise and edit this report.

I also looked through GeeksforGeeks explanations and the documentation of the relevant Python libraries while trying to understand Gaussian fitting and the functions used in the code.

I wrote and ran the Python analysis, inspected the file and branch names, produced the plots, selected the fitting interval, reproduced the numerical outputs using my own code, and decided where to stop the project.

The final responsibility for the code, reported numbers and limitations is mine.

Data, code and sources

Tools used:

Python 3.11, uproot, NumPy, Matplotlib and SciPy.

Dataset:

- $B^\pm \rightarrow J/\psi(\rightarrow \mu^+\mu^-)K^\pm$ Ntuples 4530 — 2017 Magnet Up
- Parent collection DOI: [10.7483/OOPENDATA.LHCB.EBYF.C7OY](https://doi.org/10.7483/OOPENDATA.LHCB.EBYF.C7OY)

Analysis used as a reference:

- REANA example: LHCb Run 2 $B^\pm \rightarrow J/\psi K^\pm$ analysis

Useful documentation:

- [uproot documentation](#)
- NumPy documentation: `numpy.histogram`
- SciPy documentation: `scipy.optimize.curve_fit`
- SciPy documentation: normal distribution

Code:

The complete Python code will be made available on GitHub and linked through the Data Analysis page of my website.

This is a learning project, not a peer-reviewed particle-physics study. It documents my first attempt at opening an LHCb ROOT ntuple, extracting one branch, plotting its distribution and fitting a simplified peak model.